

↳ To enable machines to solve problems systematically, a structured/systematic approach is required that considers all possibilities.

↳ Search problems define these possibilities, and search algorithms are designed to systematically explore them to find solutions.

Search

↳ Consider the problem of designing agents in a fully observable, deterministic, discrete known environment.

↳ The agent must find a sequence of actions that reaches the goal.

↳ The performance measure is defined by:

- Reaching the goal
- How expensive the path to that goal is

Search problem Components/Formulation:

1. Initial state

2. Actions $\text{Actions}(s: \text{state})$

3. Transition model $\text{Result}(s: \text{state}, a: \text{action}) \rightarrow \text{resulting state } s'$

4. Goal state set of one or more goals

5. Path cost $\text{Action-cost}(s: \text{state}, a: \text{action}, s': \text{new state})$
value $\leftarrow$

↳ The optimal solution is the sequence of actions that gives the lowest path cost for reaching the goal.

Example,

↳ On a vacation in Romania; currently in Arad.
Flight leaves tomorrow from Bucharest

1. Initial state: Arad
2. Actions: go from one city to another
3. Transition model: If you go from A to B, you end up in B.
4. Goal state: Bucharest
5. Path cost: Sum of edge costs (Total distance travelled)

State space

- ↳ The initial state, actions & transition model define the state space of the problem
- ↳ The set of all states reachable from an initial state by any sequence of actions

World State

- ↳ Covers every last detail of the environment.

Search state

- ↳ keeps only the details needed for planning

↳ Assume;

- Total Agent positions: 120
- Total food count: 30
- Ghost positions: 12
- Agent facing: NSEW
- Num ghosts: 2

Example

World state would:

$$120 \times 2^{30} \times 12^2 \times 4$$

Example,

1. States for pathing
120

2. States for cat-all-dots

$$120 \times 2^{30}$$

Abstraction:

"Process of removing irrelevant details to create abstract representation"

Example,

For navigation, we need to solve the map problem,

- States(nodes): Cities
- Actions(links/operators): Major roads

I worry about refueling, scenery etc later.

↳ We must create a simplified model of the world for the computer to deal with.

Example:

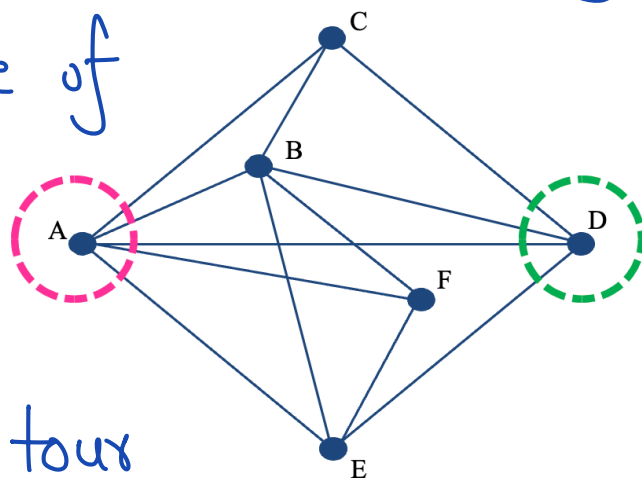
Travelling salesperson problem

↳ Find shortest tour that visits all cities without visiting any city twice & return to starting point

- States: Sequence of cities visited

- $S_0 = A$

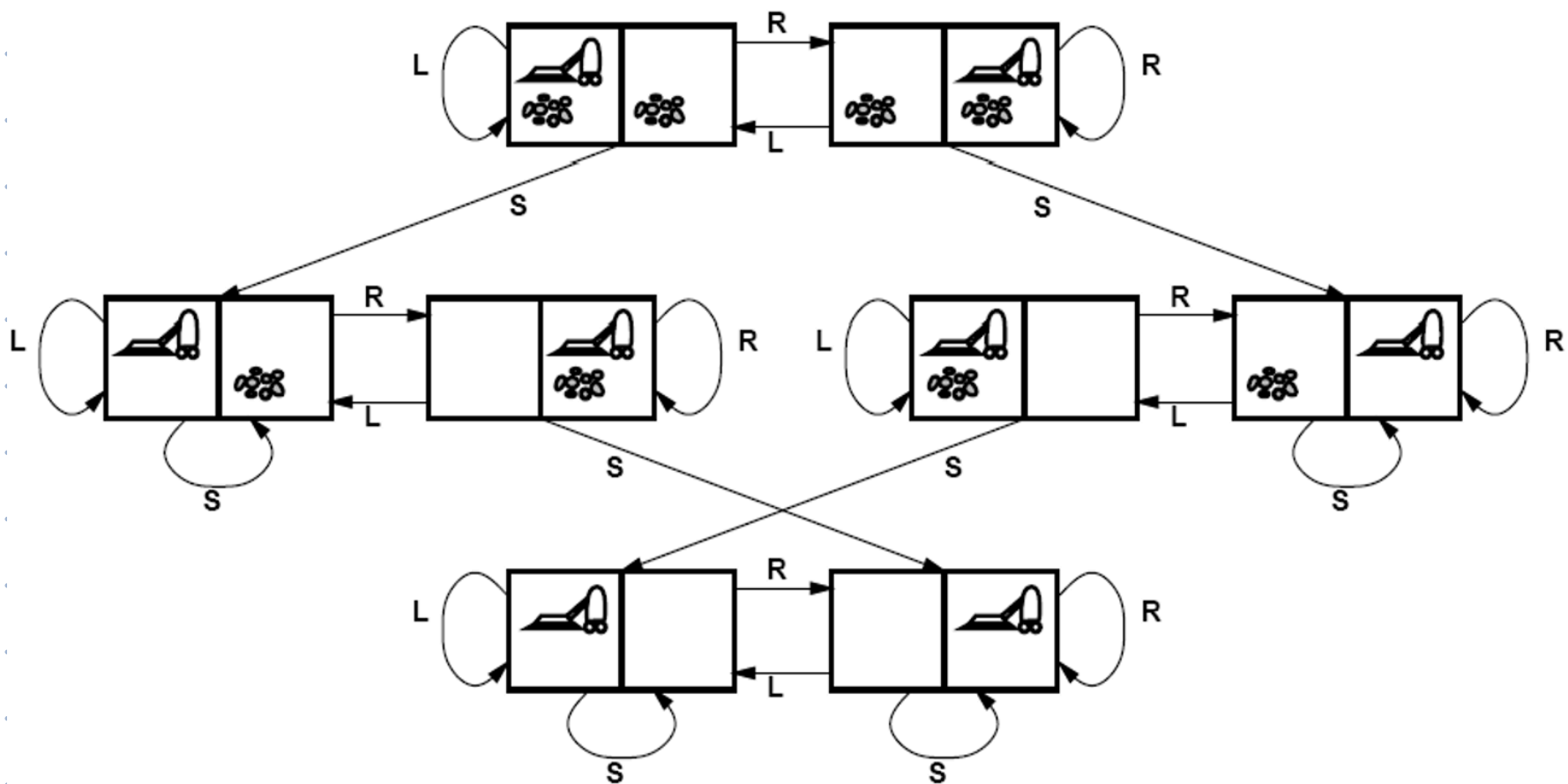
- G = a complete tour



Vacuum world

- States: Agent location & dirt location
- Actions: Left, right, suck
- Transition model

State graph



The 8-puzzle

- States: location of tiles

8-puzzle: $9!/2$

15-puzzle: $15!/2$

24-puzzle: $24!/2$

- Actions: Move blank L, R, U, D

- Path cost

1 per move

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

How to represent State Space?

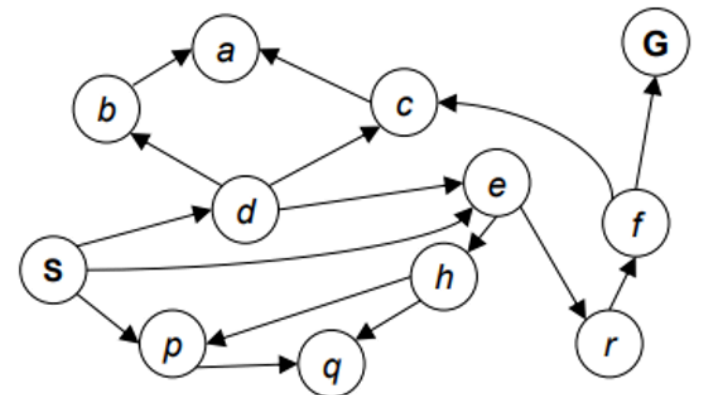
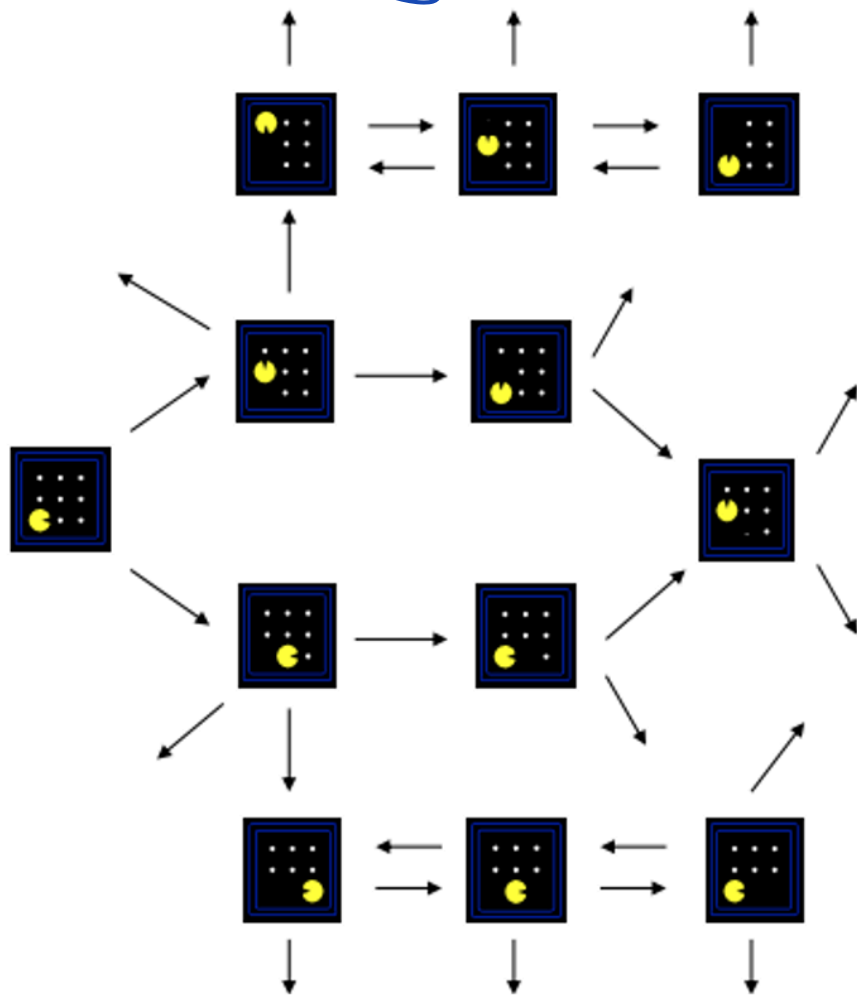
Representing search

- Represented as a directed graph where nodes are states & edges are actions.

State space graph:

A mathematical representation of a search problem

- Nodes are world configuration
- Arcs represent successors (action results)
- The goal test is set of goal nodes (maybe only one)



Tiny state space graph for a tiny search problem

↳ We search for a solution by building a search tree

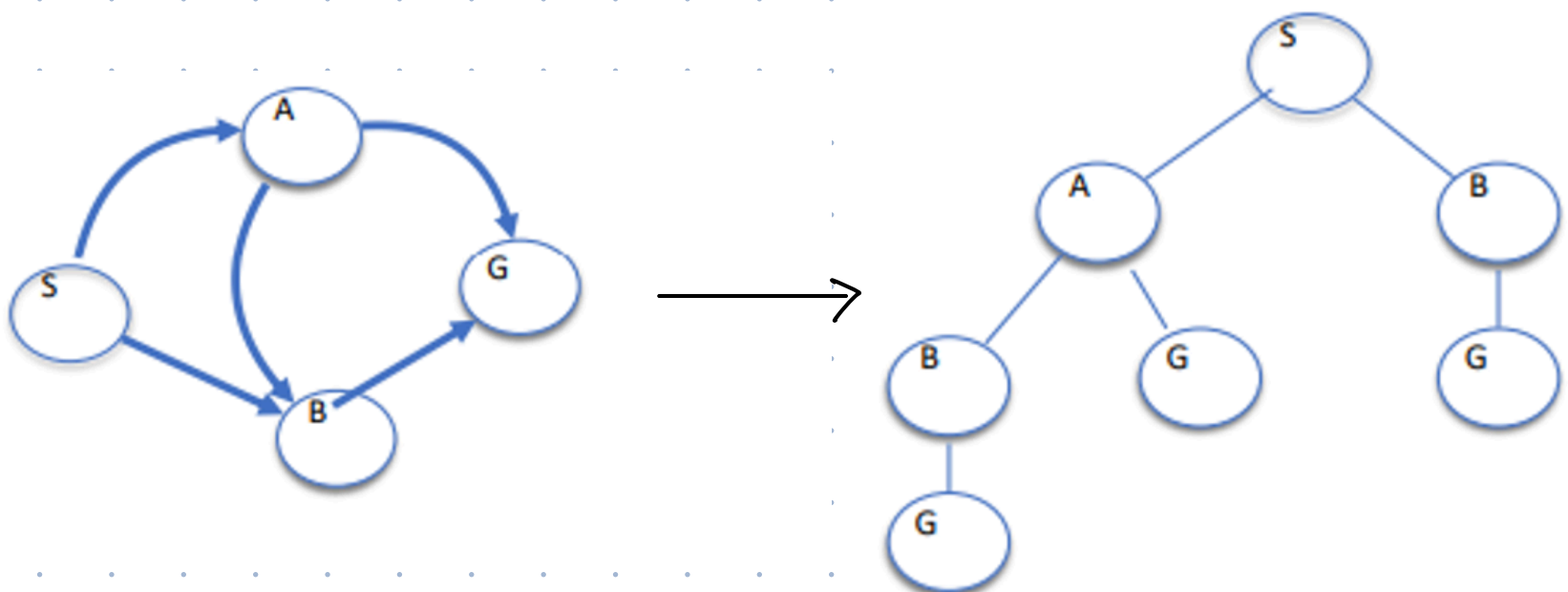
Search tree

↳ Root node = initial state

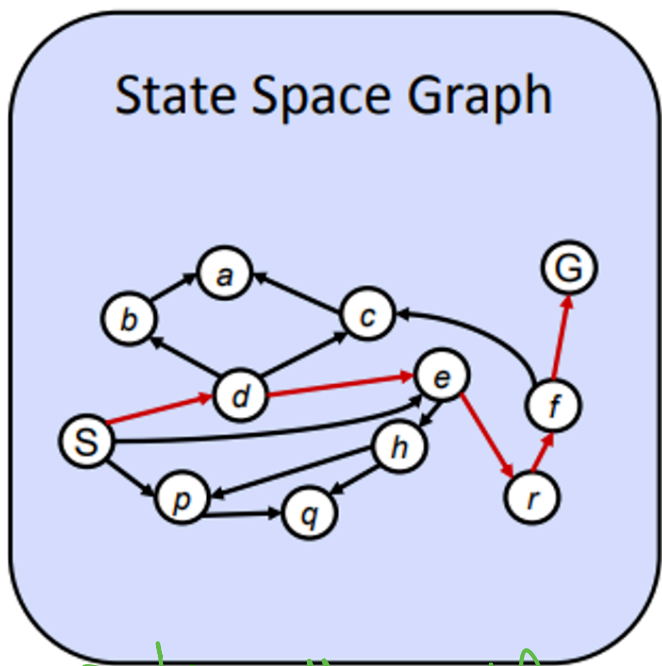
↳ Child nodes represent successor state to parent node's state.

↳ A path through the tree corresponds to a sequence of actions.

↳ Solution is path ending in goal state

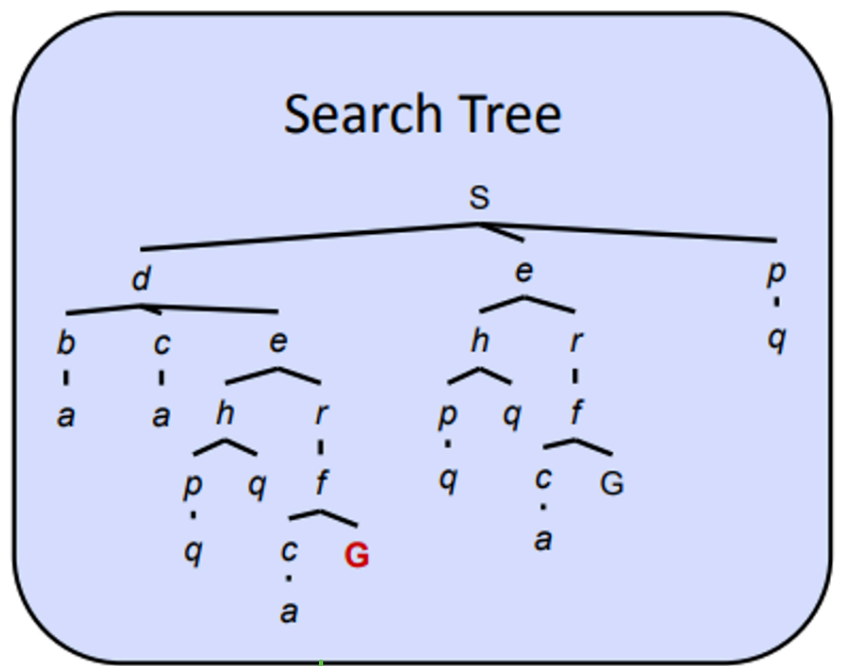


State space graphs vs Search trees



Each NODE in the search tree is an entire PATH in the state space graph.

We construct both on demand – and we construct as little as possible.



Defines the process (all path the algorithm explores)

State vs node + transitions

State is physical configuration

Node is a data structure constituting a part of a search tree contains info such as:

Example,

A child of node by action a has

state: $\text{result}(\text{node.state}, a)$

parent: node

action: a

path-cost: node.path-cost

+ $\text{action-cost}(\text{node.state}, a, \text{self.state})$

Search State: Basic idea

Let's begin at the start state & expand it by making a list of all possible successor states.

Maintain a frontier/list of unexpanded states

(Unexplored discovered states (A queue))

At each step, pick a state from the frontier to expand

(Remove one state -> expand)

Repeat until goal state is reached

↳ Try to expand as few states as possible.

Search Tree

↳ A data structure that represents exploration of a search problem

↳ Built from state space graph

Tree search

↳ A search algorithm

↳ It systematically traverses the tree, considering different paths by choosing to expand & visit next.

Handling repeating states

- Initialize the frontier/fringe using the starting state
- While the frontier is not empty,
 - Choose a frontier/fringe node according to a search strategy & take it off the frontier/fringe
 - If the node contains the goal state, return the solution
 - Else expand the node & add its children to the frontier.
- To handle repeated states:
 - Everytime you expand a node, add that state to the explored set; do not put explored sets on the frontier again.
 - Every time you add a node to the frontier, check whether it already exists in the frontier with a higher path cost, if yes, replace that node with the new one.